

Global.asax

In an ASP.NET MVC application, the Global.asax file (also known as the ASP.NET Application File) is used for application-level events and configuration. It allows developers to write code that responds to global events raised by ASP.NET or the HTTP modules.

Here's an overview of its purpose and common uses:

Purpose of Global.asax:

The Global.asax file allows you to define event handlers for application-level events, such as:

- Application start and end
- Session start and end
- Error handling for unhandled exceptions
- Custom logic for requests

The file is automatically created in ASP.NET MVC projects and is particularly useful for performing initialization tasks when the application first loads or handling unexpected errors.

```
protected void Application_Start()
{
    AreaRegistration.RegisterAllAreas();
    RouteConfig.RegisterRoutes(RouteTable.Routes);
    FilterConfig.RegisterGlobalFilters(GlobalFilters.Filters);
    // Any other initialization logic here
}
```

RouteConfig

In an ASP.NET MVC application, the `RouteConfig.cs` file is used to define URL routing rules for the application. Routing is essential in MVC as it allows mapping of URLs to specific controllers and actions. By defining routes, you can control how URLs are structured and how they correspond to your controllers, actions, and parameters.

Location and Purpose of `RouteConfig.cs`

- **Location:** The `RouteConfig.cs` file is usually located in the `App_Start` folder.
- **Purpose:** This file sets up the routes that ASP.NET MVC uses to handle requests. When the application starts, the `RouteConfig.RegisterRoutes()` method is called (usually in `Global.asax`), and it adds routes to the `RouteTable`.

```
using System.Web.Mvc;
using System.Web.Routing;

public class RouteConfig
{
    public static void RegisterRoutes(RouteCollection routes)
    {
        routes.IgnoreRoute("{resource}.axd/{*pathInfo}");

        // Default route
        routes.MapRoute(
            name: "Default",
            url: "{controller}/{action}/{id}",
            defaults: new { controller = "Home", action = "Index", id =
UrlParameter.Optional }

        );
    }
}
```

ActionResult Type	Description	Example URL
ViewResult	Renders a view based on a parameter.	/Controller/Details/5
PartialViewResult	Renders a partial view using parameters.	/Controller/ProductInfo?productId=5
JsonResult	Returns JSON data based on parameters (e.g., for AJAX).	/Controller/GetCustomerData?customerId=42
RedirectResult	Redirects to a specific URL with parameters.	Redirects to /Profile/View/7
RedirectToActionResult	Redirects to another action within the app, passing parameters.	Redirects to /User/Profile/7
RedirectToRouteResult	Redirects to a route with route values as parameters.	Redirects to /Products/Details/5
ContentResult	Returns raw text, HTML, or other content based on parameters.	/Controller/GetMessage/5
FileResult	Returns a file for download, using parameters to find the file.	/Controller/DownloadFile/3
JavaScriptResult	Returns JavaScript code to the client based on a parameter.	/Controller/ShowAlert/4
EmptyResult	Returns an empty response, performing no action.	/Controller/DoNothing/5

```

public ActionResult Details(int id)
{
    var item = _repository.GetItemById(id);
    return View(item);
}

public PartialViewResult ProductInfo(int productId)
{
    var product = _repository.GetProductById(productId);
    return PartialView("_ProductInfoPartial", product);
}

public JsonResult GetCustomerData(int customerId)
{
    var customerData = _repository.GetCustomerById(customerId);
    return Json(customerData, JsonRequestBehavior.AllowGet);
}

```

```

}

public RedirectResult RedirectToProfile(int userId)
{
    return Redirect($" /Profile/View/{userId}");
}

public RedirectToActionResult GoToProfile(int userId)
{
    return RedirectToAction("Profile", "User", new { id = userId });
}

public RedirectToRouteResult RedirectToCustomRoute(int categoryId, string
actionName)
{
    return RedirectToRoute("Default", new { controller = "Products", action =
actionName, id = categoryId });
}

public ContentResult GetMessage(int id)
{
    string message = $"Message for item with ID: {id}";
    return Content(message);
}

public FileResult DownloadFile(int fileId)
{
    var fileData = _repository.GetFilesByFileId(fileId);
    return File(fileData.Content, fileData.ContentType, fileData.FileName);
}

public JavaScriptResult ShowAlert(int alertId)
{
    var alertMessage = _repository.GetAlertMessageById(alertId);
    return JavaScript($"alert('{alertMessage}');");
}

```

Razor engine

The Razor engine is a templating engine used in ASP.NET to generate dynamic web pages with HTML and C# or VB.NET code. It's specifically designed to work with ASP.NET MVC, ASP.NET Core, and Blazor applications, allowing developers to write server-side code within their HTML markup seamlessly. Razor views typically use the `.cshtml` or `.vbhtml` file extension.

Key Features of Razor

1. **Syntax Simplicity:**
 - o Razor syntax is clean and concise. By using the `@` symbol to transition between HTML and C# code, Razor makes it easy to embed server-side logic directly into HTML.
2. **C# or VB.NET Embedded Code:**
 - o The Razor engine can interpret C# (or VB.NET) expressions, loops, and conditionals within HTML. This enables dynamic content generation based on server-side logic.
3. **Automatic Encoding:**
 - o Razor automatically encodes HTML output to prevent injection attacks. For example, `@Model.Name` will be HTML-encoded by default.
4. **Intelligent Code Rendering:**
 - o The `@` symbol is used to switch from HTML to C# code, and it's context-sensitive, so Razor knows when to return to HTML.

Common Razor Syntax Elements

1. **Variables and Expressions:**
 - o You can display variables and expressions using `@`.

```
<p>Hello, @Model.UserName!</p>
```

```
<p>Current year: @DateTime.Now.Year</p>
```

2. **Code Blocks:**

- For multiple lines of code, use `@{ ... }`.

```
@{  
    var welcomeMessage = "Welcome to Razor!";  
}  
  
<p>@welcomeMessage</p>
```

3. Conditionals

You can use `if`, `else`, `switch`, etc.

```
@if (Model.IsLoggedIn)
{
    <p>Welcome, @Model.UserName!</p>
}
else
{
    <p>Please log in.</p>
}
```

4. Loops:

`for`, `foreach`, `while`, and other loops are supported.

```
<ul>
    @foreach (var item in Model.Items)
    {
        <li>@item.Name</li>
    }
</ul>
```

5. Helper Methods:

Razor supports custom helper methods for code reusability.

```
@helper FormatPrice(decimal price)
{
    <span class="price">${@price.ToString("F2")}</span>
}
@FormatPrice(29.99M)
```

6. Razor View Example

Here's a simple example of a Razor view that displays a list of items using an `@model` directive.

```
@model List<string>
<h2>Item List</h2>
<ul>
    @foreach (var item in Model)
    {
        <li>@item</li>
    }
</ul>
```

Advantages of Using Razor

- **Lightweight and Fast:** Razor views are parsed and rendered quickly.
- **Readable and Maintainsable:** The Razor syntax is concise and easy to read.
- **Seamless C# Integration:** Razor allows for robust C# code integration in views, enabling logic and data processing without JavaScript or other client-side code.

Razor in ASP.NET Core

In ASP.NET Core, Razor is also used to build pages in the **Razor Pages** framework. Razor Pages is a page-centric model that uses Razor syntax for building web applications in a more simplified structure than MVC.